

MATH 7243 — Lab 2: Methods in Linear Regression

Wilson Glass

October 3, 2026

Problem 1: Bootstrapping a Confidence Interval

Setup

We use the Ames housing data, cleaned as in the Master notebook: remove the outliers with $\text{GrLivArea} + \text{BsmtUnfSF} \geq 4000$, keep only the numerical columns, and drop the categorical `MSSubClass`. This leaves $n = 1436$ houses. We fit the one-variable model

$$\text{SalePrice} = \beta_0 + \beta_1 \cdot \text{1stFlrSF}.$$

```
import io
import requests
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt

url = "https://raw.githubusercontent.com/tipthederiver/Math-7243-2020/master/Datasets/Ames/train.csv"
ames = pd.read_csv(io.StringIO(requests.get(url).content.decode('utf-8'))))

z = ames['GrLivArea'] + ames['BsmtUnfSF'] < 4000
data = ames[z]
data = data.select_dtypes(include=['int64', 'float64'])
data = data.drop(columns='MSSubClass') # 1436 rows
```

To bootstrap, we draw a sample of $N = 1436$ rows from the data *with replacement*, fit $\beta = (X^T X)^{-1} X^T y$ on that sample, and store β_0 and β_1 . Repeating this 1000 times gives 1000 estimates of each parameter.

```
N = 1000
beta0 = np.zeros(N)
beta1 = np.zeros(N)

for i in range(N):
    # 1. resample rows of the data, with replacement
    sample = data.sample(n=len(data), replace=True)

    # 2. build X (with a column of 1's) and Y
    X = np.matrix(sample["1stFlrSF"]).reshape(-1, 1)
    Y = np.matrix(sample["SalePrice"]).reshape(-1, 1)
    Xa = np.append(np.ones(X.shape), X, 1)
```

```

# 3. fit with the normal equation
betas = (Xa.T*Xa).I*Xa.T*Y

# 4. store
beta0[i] = betas[0, 0]
beta1[i] = betas[1, 0]

```

Part 1: Histograms of β_0 and β_1

```

fig, axes = plt.subplots(1, 2)
fig.set_size_inches(12, 4)

axes[0].hist(beta0, bins=30)
axes[0].set_title("Bootstrap distribution of  $\beta_0$  (intercept)")
axes[0].set_xlabel(" $\beta_0$ "); axes[0].set_ylabel("Count")

axes[1].hist(beta1, bins=30)
axes[1].set_title("Bootstrap distribution of  $\beta_1$  (slope)")
axes[1].set_xlabel(" $\beta_1$ "); axes[1].set_ylabel("Count")

plt.show()

```

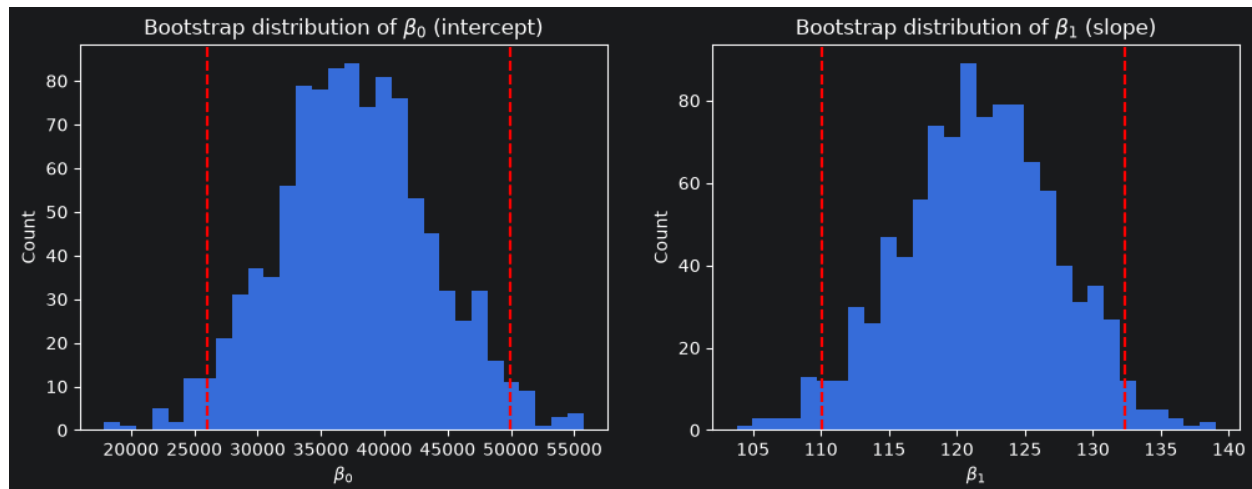


Figure 1: Bootstrap distributions of β_0 and β_1 over 1000 resamples. The red dashed lines mark the bootstrap 95% confidence interval (Part 2).

Both distributions are roughly bell-shaped, centered near the full-data estimates $\hat{\beta}_0 \approx 37,566$ and $\hat{\beta}_1 \approx 121.8$. Their spread shows how much the fitted line would change if we had drawn a different sample of houses.

Part 2: Bootstrap 95% Confidence Interval

After sorting, the middle 950 of the 1000 values run from index 25 to index 974.

```

beta0.sort()
beta1.sort()

```

```
print("95% bootstrap CI for beta0:", beta0[25], "to", beta0[974])
print("95% bootstrap CI for beta1:", beta1[25], "to", beta1[974])
```

Answer:

$$\beta_0 \in [26,000.33, 49,953.89]$$

$$\beta_1 \in [110.09, 132.32]$$

Part 3: Confidence Interval from the Formula

From Section 4 of the notes, the covariance matrix of the least-squares estimate $\hat{\theta} = (\hat{\beta}_0, \hat{\beta}_1)$ is

$$\text{Cov}(\hat{\theta}) = \hat{\sigma}^2 (X^T X)^{-1}, \quad \hat{\sigma}^2 = \frac{\text{RSS}}{n - 2},$$

where $\hat{\sigma}^2$ estimates the variance of the noise around the regression line, and we divide by $n - 2$ because two parameters were fit. The diagonal entries of $\text{Cov}(\hat{\theta})$ are $\text{Var}(\hat{\beta}_0)$ and $\text{Var}(\hat{\beta}_1)$. Their square roots are the standard errors, and the 95% confidence interval is $\hat{\beta}_j \pm 1.96 \text{SE}(\hat{\beta}_j)$. Here we use all 1436 rows.

```
X = np.matrix(data["1stFlrSF"]).reshape(-1, 1)
Y = np.matrix(data["SalePrice"]).reshape(-1, 1)
Xa = np.append(np.ones(X.shape), X, 1)
n = len(Y)

betas = (Xa.T*Xa).I*Xa.T*Y
RSS = ((Y - Xa*betas).T*(Y - Xa*betas))[0, 0]
sigma2 = RSS/(n - 2) # variance of the noise

var_betas = sigma2*(Xa.T*Xa).I # Cov(theta) = variance * (X^T X)^-1
se = np.sqrt(np.diag(var_betas)) # standard errors

print("Formula 95% CI for beta0:", betas[0, 0] - 1.96*se[0], "to", betas[0, 0] + 1.96*se[0])
print("Formula 95% CI for beta1:", betas[1, 0] - 1.96*se[1], "to", betas[1, 0] + 1.96*se[1])
print("Bootstrap std of beta0:", np.std(beta0), " formula SE:", se[0])
print("Bootstrap std of beta1:", np.std(beta1), " formula SE:", se[1])
```

Answer:

$$\beta_0 \in [27,673.93, 47,457.58]$$

$$\beta_1 \in [113.52, 129.99]$$

Comparison

	95% CI		Standard error	
	Formula	Bootstrap	Formula	Bootstrap
β_0	[27,674, 47,458]	[26,000, 49,954]	5,047	6,088
β_1	[113.52, 129.99]	[110.09, 132.32]	4.20	5.74

The two methods agree on the center of each interval, but the bootstrap intervals are wider: about 21% wider for β_0 and 35% wider for β_1 . The formula assumes the noise has the same variance σ^2 for every house. That assumption does not hold here: prices of large houses vary much more than prices of small ones, so the scatter of `SalePrice` against `1stFlrSF` fans out as `1stFlrSF` increases. The formula therefore underestimates the uncertainty in the fit. The bootstrap makes no assumption about the noise. It resamples the actual data, so it captures this extra variability, and its intervals are the more trustworthy of the two for this data.

Problem 2: Linear Methods on High-Dimensional Data

Setup

The MRI dataset contains 609 brain slice images, each $176 \times 176 = 30,976$ pixels. Each image is converted to grayscale (averaging the color channels), flattened into a single row, and downsampled by keeping every 8th pixel. This gives a data matrix of shape 609×3872 . The **features** are the 3872 pixel intensities and the **target** is the normalized whole-brain volume **nWBV** from **labels.csv**. The other columns of **labels.csv** (age, eTIV, ASF, ...) are not used, since the goal is to predict nWBV from the images alone.

```
import matplotlib
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

file_dir = 'data/'
labels = pd.read_csv(file_dir + 'labels.csv')

DS = 8 # downsample rate
im_size = int(30976/DS) # 3872 features

data = np.zeros([609, im_size])
for i, file_name in enumerate(labels.Filename):
    img = np.mean(matplotlib.image.imread(file_dir + file_name), axis=2).reshape(-1)
    data[i, :] = img[::DS]

X = data
y = labels["nWBV"]

x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=.2, random_state=255
)
```

The 80/20 split gives 487 training images and 122 test images. With 3872 features and only 487 training samples, there are more unknowns than equations, so ordinary least squares can fit the training data perfectly and generalizes poorly. This is why regularization is needed. Every model below is scored by its r^2 on the *test* set. Training r^2 always improves as the penalty goes to zero, so it cannot detect overfitting.

Part 1: Best α for Ridge Regression

We use statsmodels' `fit_regularized` with `L1_wt=0` (pure ridge), which minimizes

$$\frac{1}{2n} \text{RSS} + \frac{\alpha}{2} \sum_i \beta_i^2.$$

Because many background pixels are constant, `add_constant` needs `has_constant="add"`. Otherwise it skips adding the intercept column on the test set.

The search was done in two stages. A coarse sweep over $\alpha = 10^{-6}, 10^{-5}, \dots, 10^3$ (`np.logspace(-6, 3, 10)`) showed the test r^2 peaking near $\alpha = 10^{-2}$:

α	10^{-3}	10^{-2}	10^{-1}	10^0	10^1
test r^2	0.284	0.398	0.285	0.089	-0.247

A fine sweep of 50 values between 0.001 and 0.05 then located the maximum:

```
import statsmodels.api as sm

alphas = np.linspace(0.001, 0.05, 50)
r_sqds = []

for alp in alphas:
    ols = sm.OLS(y_train, sm.add_constant(x_train, has_constant="add"))
    ols_result = ols.fit_regularized(L1_wt=0, alpha=alp)
    pred = ols_result.predict(sm.add_constant(x_test, has_constant="add"))
    r_sqds.append(r2_score(y_test, pred))

plt.plot(alphas, r_sqds, "o-")
plt.xlabel("alpha"); plt.ylabel("test $r^2$")
plt.show()

best = np.argmax(r_sqds)
print("Best alpha:", alphas[best], "with test $r^2$ =", r_sqds[best])
```

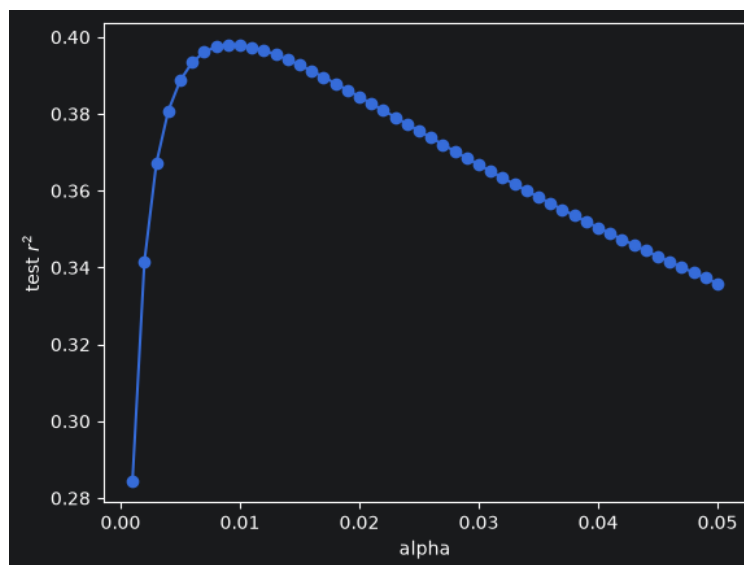


Figure 2: Ridge regression: test r^2 against α (fine sweep).

Answer: The best ridge penalty is $\alpha \approx 0.009$, with test $r^2 \approx 0.398$.

Justification. The test r^2 curve rises to a single peak and then falls, which is the bias–variance tradeoff:

- **α too small:** the penalty is too weak to stop the model from fitting noise among 3872 pixel coefficients with only 487 training images. Test r^2 is 0.284 at $\alpha = 0.001$ and only 0.063 at $\alpha = 0$.
- **α too large:** the penalty shrinks all coefficients so far toward zero that the model can no longer capture the real relationship. Test r^2 falls steadily past the peak, to 0.336 at $\alpha = 0.05$, and becomes negative by $\alpha = 10$. A negative r^2 means the model does worse than predicting the mean nWBV for every image.

- $\alpha \approx 0.009$ balances the two and gives the highest test r^2 .

The peak is fairly flat. Every α from about 0.006 to 0.016 gives test $r^2 \geq 0.39$, so the best value is better described as “about 0.01” than as exactly 0.009. This α is on statsmodels’ scale. sklearn’s `Ridge` drops the $\frac{1}{2n}$ factor on the RSS, so its optimal α would be numerically different.

Part 2: Best λ for Lasso Regression

Notation. Here λ is the lasso penalty strength, as in the lasso loss

$$\frac{1}{2n} \text{RSS} + \lambda \sum_i |\beta_i|.$$

In sklearn's `Lasso` (and statsmodels' `fit_regularized`) this parameter is named `alpha`, so each λ is passed in as `alpha=lam`. In the lab's elastic-net notation, pure lasso fixes the mixing weight `L1_wt = 1`, which leaves the penalty strength as the only parameter to tune.

We use sklearn's `Lasso`, which minimizes exactly the loss above (the same $\frac{1}{2n}$ scaling as statsmodels), so the values are directly comparable. statsmodels' lasso solver runs coordinate descent in pure Python and was too slow with 3872 features. At the smallest λ values the solver does not fully converge (`ConvergenceWarning`). Those models overfit and are not candidates for the best λ , so the warnings are suppressed.

Coarse sweep

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.linear_model import Lasso

warnings.filterwarnings("ignore", category=ConvergenceWarning)

lambdas = np.logspace(-6, 3, 10)
r_sqds = []
n_nonzero = []

for lam in lambdas:
    lasso = Lasso(alpha=lam, max_iter=10_000)
    lasso.fit(x_train, y_train)
    r_sqds.append(r2_score(y_test, lasso.predict(x_test)))
    n_nonzero.append(np.sum(lasso.coef_ != 0)) # pixels kept

plt.semilogx(lambdas, r_sqds, "o-")
plt.xlabel("$\\lambda$ (lasso penalty strength)"); plt.ylabel("test $r^2$")
plt.show()
```

λ	10^{-6}	10^{-5}	10^{-4}	10^{-3}	$\geq 10^{-2}$
test r^2	0.004	0.200	0.366	-0.048	-0.048
pixels kept	496	377	77	0	0

Zooming in

The coarse sweep puts the peak near $\lambda = 10^{-4}$. A first zoom over 30 values from 10^{-5} to 10^{-3} placed the best value at $\lambda \approx 4.4 \times 10^{-5}$. Its grid points are 3.4×10^{-5} apart, which is wide compared with λ itself near the peak, so a second zoom over 13 values from 2×10^{-5} to 8×10^{-5} was used to pin it down. Both zooms use the same loop as above with `np.linspace`:

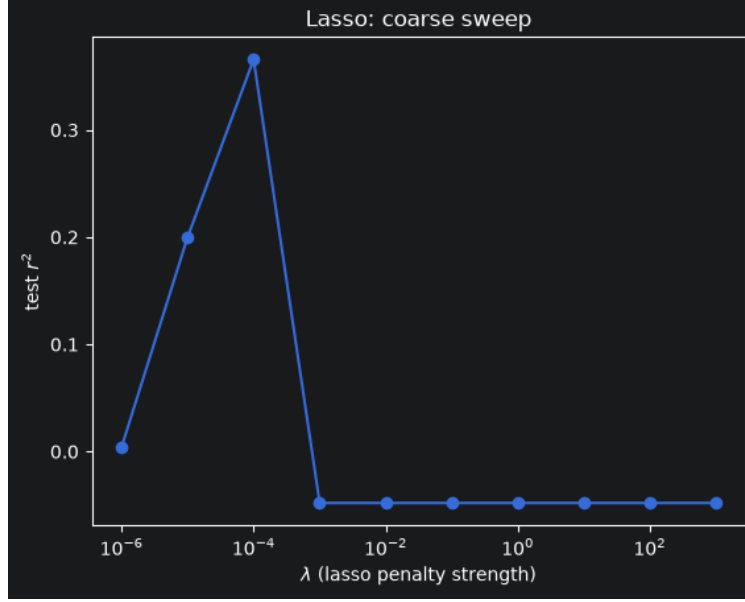


Figure 3: Lasso: test r^2 against λ , coarse sweep (log scale).

```
lambdas = np.linspace(0.00001, 0.001, 30) # zoom 1
lambdas = np.linspace(0.00002, 0.00008, 13) # zoom 2
```

$\lambda (\times 10^{-5})$	2.0	3.0	4.0	5.0	5.5	6.0	7.0
test r^2	0.320	0.356	0.378	0.389	0.389	0.389	0.381
pixels kept	300	249	202	168	148	135	105

Answer: The best lasso penalty is $\lambda \approx 5.5 \times 10^{-5}$, with test $r^2 \approx 0.389$. At this λ , lasso keeps only **148 of the 3872 pixels** (about 4%) and sets the rest exactly to zero.

Justification.

- **λ too small:** the penalty is too weak, so lasso keeps hundreds of pixels and overfits the 487 training images. At $\lambda = 10^{-6}$ it keeps 496 pixels and the test r^2 is only 0.004. At $\lambda = 10^{-5}$ it keeps 377 pixels with test $r^2 = 0.200$. The solver also fails to converge here, a sign that too many correlated pixels are competing to explain the same signal.
- **λ too large:** the penalty sets more and more coefficients to exactly zero. By $\lambda \approx 5 \times 10^{-4}$ only about 10 pixels remain and test r^2 has fallen to 0.03. For $\lambda \geq 8 \times 10^{-4}$ every pixel is dropped, the model predicts the mean nWBV for every image, and test $r^2 = -0.048$.
- $\lambda \approx 5.5 \times 10^{-5}$ balances the two. It discards the pixels that do not help and keeps a small set that predicts brain volume.

As with ridge, the peak is flat: λ from 5×10^{-5} to 6×10^{-5} all give test $r^2 \approx 0.389$, keeping between 135 and 168 pixels.

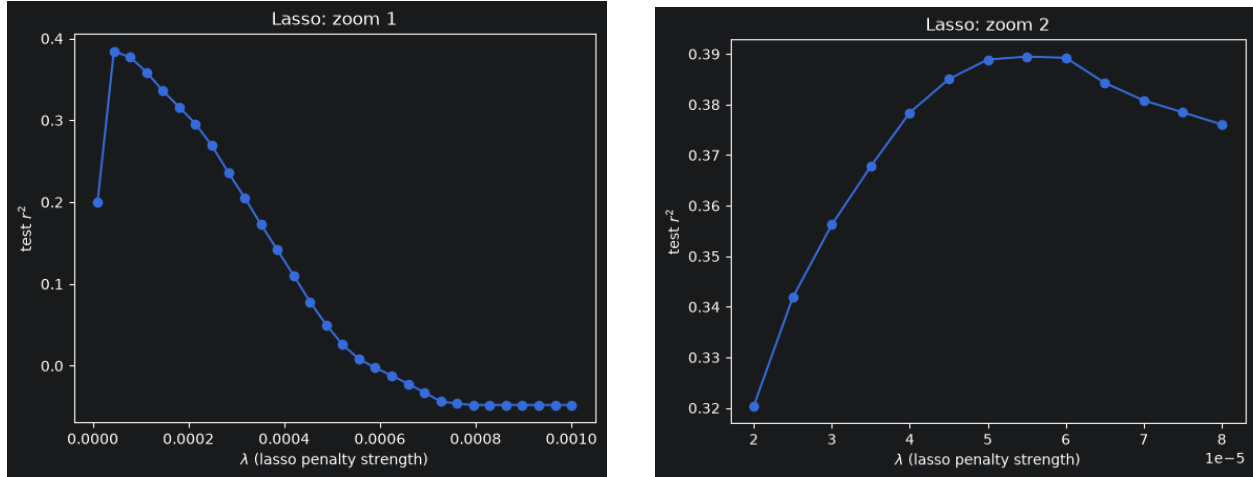


Figure 4: Lasso: test r^2 against λ . Left: zoom 1 (10^{-5} to 10^{-3}). Right: zoom 2 (2×10^{-5} to 8×10^{-5}).

Comparison with ridge

Method	Best penalty	Test r^2	Pixels used
Ridge	$\alpha \approx 0.009$	0.398	3872 (all, shrunk)
Lasso	$\lambda \approx 5.5 \times 10^{-5}$	0.389	148

Lasso nearly matches ridge's accuracy with a far simpler model: about 4% of the pixels. Ridge's small edge fits how brain volume appears in an MRI slice. It shows up as the overall amount of tissue against dark space across the whole image rather than in a few specific pixels. Spreading small weights over every pixel (ridge) suits this kind of diffuse signal slightly better than selecting a few pixels (lasso).